

Day 10: JDBC & Database Connectivity (19-June-2026, Friday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 10, learners will be able to:

- Understand JDBC architecture and the role of JDBC drivers.
- Connect to MySQL database using `DriverManager` and `Connection` .
- Execute CRUD operations using `Statement` and `PreparedStatement` .
- Use `PreparedStatement` to prevent SQL injection and handle dynamic parameters.
- Manage database transactions with `commit()` and `rollback()` .
- Handle SQL exceptions properly using `SQLException` .
- Solve LeetCode: *Rotate Array* and *Move Zeroes* (algorithmic problems, not DB-related).
- Solve HackerRank: *Java JDBC Concepts* (typically conceptual or simple queries).
- Design an **Enterprise Employee Database Management System** with JDBC.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 9 & Day 10 Overview – JDBC Introduction	15 min
9:15 – 9:45	JDBC Architecture – DriverManager, Connection, Statement, ResultSet	30 min
9:45 – 10:15	JDBC Driver Types and Database Setup (MySQL)	30 min
10:15 – 10:30	Connecting to Database – Loading Driver, Getting Connection	15 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:30	Statement Interface – executeQuery, executeUpdate, ResultSet	45 min

11:30 – 12:15	PreparedStatement – Parameters, Performance, SQL Injection Prevention	45 min
12:15 – 12:30	Hands-on: Statement vs PreparedStatement Comparison	15 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:15	CRUD Operations – Create, Read, Update, Delete (with examples)	45 min
2:15 – 2:45	Transactions – commit, rollback, Savepoint, Auto-commit	30 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:30	LeetCode: Rotate Array & Move Zeroes (Approach Discussion)	30 min
3:30 – 3:45	HackerRank: Java JDBC Concepts Walkthrough	15 min
3:45 – 4:15	Corporate Use Case: Enterprise Employee Database Management System	30 min
4:15 – 4:30	Homework & Closing	15 min

1. JDBC Overview

1.1 What is JDBC?

- **Java Database Connectivity (JDBC)** is a Java API for connecting and executing queries with databases.
- Provides a standard interface to interact with relational databases (MySQL, PostgreSQL, Oracle, etc.).
- Part of `java.sql` and `javax.sql` packages.

1.2 JDBC Architecture

```

Java Application
    ↓
JDBC API (java.sql.*)
    ↓
JDBC Driver Manager
    ↓
JDBC Driver (Vendor-specific)
    ↓
Database (MySQL/Oracle/PostgreSQL)

```

1.3 Key Components

Component	Description
DriverManager	Manages JDBC drivers; establishes database connections.
Connection	Represents a session with the database.
Statement	Used to execute static SQL queries.
PreparedStatement	Executes parameterized queries (precompiled).
CallableStatement	Executes stored procedures.
ResultSet	Represents result set of a query (table of data).
SQLException	Handles database errors.

2. JDBC Drivers

2.1 Types of JDBC Drivers

Type	Description	Example
Type 1: JDBC-ODBC Bridge	Translates JDBC calls to ODBC. Deprecated.	<code>sun.jdbc.odbc.JdbcOdbcDriver</code>
Type 2: Native-API Driver	Uses database client libraries (platform-dependent).	Oracle OCI driver
Type 3: Network Protocol Driver	Middleware server translates JDBC to DB protocol.	Middleware-based
Type 4: Thin Driver (Pure Java)	Directly communicates with DB using native protocol.	MySQL Connector/J, PostgreSQL JDBC

2.2 MySQL Connector/J

- Type 4 driver for MySQL.
- Download: `mysql-connector-java-x.x.x.jar`.
- Add to classpath:
 - IDE: Add as library.
 - Command-line: `java -cp .;mysql-connector-java.jar MyClass` (Windows) or `java -cp mysql-connector-java.jar MyClass` (Linux).

3. Connecting to Database

3.1 Loading the Driver (Optional in Java 6+)

```
// Explicit loading (older versions)
Class.forName("com.mysql.cj.jdbc.Driver");
```

- In Java 6+, DriverManager automatically loads drivers from classpath.

3.2 Getting a Connection

```
String url = "jdbc:mysql://localhost:3306/mydb";
String username = "root";
String password = "password";

try (Connection conn = DriverManager.getConnection(url, username, password)) {
    // Use connection
} catch (SQLException e) {
    e.printStackTrace();
}
```

3.3 Connection URL Format

- MySQL: `jdbc:mysql://hostname:port/databaseName`
- PostgreSQL: `jdbc:postgresql://hostname:port/databaseName`
- Oracle: `jdbc:oracle:thin:@hostname:port:SID`

3.4 Common Connection Properties

- `useSSL=false` – disable SSL (for local development).
- `serverTimezone=UTC` – set timezone (MySQL 8+).
- `allowPublicKeyRetrieval=true` – for some MySQL 8 setups.

4. Statement Interface

4.1 Creating a Statement

```
Statement stmt = connection.createStatement();
```

4.2 Executing Queries

Method	Description	Use Case
<code>executeQuery(String sql)</code>	Executes SELECT; returns ResultSet.	Read operations
<code>executeUpdate(String sql)</code>	Executes INSERT/UPDATE/DELETE; returns row count.	Write operations
<code>execute(String sql)</code>	Returns boolean (true if ResultSet, false if update count).	Unknown SQL type

4.3 ResultSet

- Represents the result of a query (cursor-based).
- **Moving cursor:** `next()`, `previous()`, `first()`, `last()`, `absolute(int)`.
- **Retrieving data:** `getInt(columnIndex)`, `getString(columnName)`, `getDouble()`, etc.

4.4 Example – SELECT

```
String sql = "SELECT id, name, salary FROM employees";
try (Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(sql)) {
    while (rs.next()) {
        int id = rs.getInt("id");
        String name = rs.getString("name");
        double salary = rs.getDouble("salary");
        System.out.println(id + " " + name + " " + salary);
    }
}
```

4.5 Example – INSERT

```
String sql = "INSERT INTO employees (name, salary) VALUES ('John', 50000)";
int rows = stmt.executeUpdate(sql);
System.out.println(rows + " row(s) inserted.");
```

5. PreparedStatement

5.1 Why PreparedStatement?

- **Prevents SQL Injection** – parameters are escaped automatically.
- **Better Performance** – query is precompiled and reused.
- **Readable Code** – separates SQL structure from data.

5.2 Creating a PreparedStatement

```
String sql = "INSERT INTO employees (name, salary) VALUES (?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "John");
pstmt.setDouble(2, 50000);
int rows = pstmt.executeUpdate();
```

5.3 Setting Parameters

Type	Method
int	setInt(int index, int value)
String	setString(int index, String value)
double	setDouble(int index, double value)
Date	setDate(int index, java.sql.Date value)
null	setNull(int index, int sqlType)
Object	setObject(int index, Object value)

5.4 PreparedStatement vs Statement

Aspect	Statement	PreparedStatement
SQL Injection	Vulnerable	Prevented
Performance	Compiles every time	Precompiled, reused
Code readability	String concatenation	Parameterized, clean
Use Case	Static queries	Dynamic queries with parameters

5.5 Example – SELECT with Parameters

```
String sql = "SELECT * FROM employees WHERE salary > ? AND department = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setDouble(1, 50000);
pstmt.setString(2, "IT");
ResultSet rs = pstmt.executeQuery();
```

5.6 Batch Updates

```
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO employees (name) VALUES (?)")
pstmt.setString(1, "Alice");
pstmt.addBatch();
pstmt.setString(1, "Bob");
pstmt.addBatch();
int[] results = pstmt.executeBatch();
```

6. CRUD Operations

6.1 CREATE (INSERT)

```
String sql = "INSERT INTO employees (name, salary, department) VALUES (?, ?, ?)";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "John");
    pstmt.setDouble(2, 75000);
    pstmt.setString(3, "IT");
    int rows = pstmt.executeUpdate();
    System.out.println("Inserted: " + rows);
}
```

6.2 READ (SELECT)

```
String sql = "SELECT * FROM employees WHERE department = ?";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "IT");
    ResultSet rs = pstmt.executeQuery();
    while (rs.next()) {
        System.out.println(rs.getInt("id") + " " + rs.getString("name"));
    }
}
```

6.3 UPDATE

```
String sql = "UPDATE employees SET salary = ? WHERE id = ?";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setDouble(1, 80000);
    pstmt.setInt(2, 1);
    int rows = pstmt.executeUpdate();
    System.out.println("Updated: " + rows);
}
```

6.4 DELETE

```
String sql = "DELETE FROM employees WHERE id = ?";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setInt(1, 5);
    int rows = pstmt.executeUpdate();
    System.out.println("Deleted: " + rows);
}
```

6.5 Obtaining Auto-Generated Keys

```
String sql = "INSERT INTO employees (name) VALUES (?)";
try (PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
    pstmt.setString(1, "Alice");
    pstmt.executeUpdate();
    ResultSet rs = pstmt.getGeneratedKeys();
    if (rs.next()) {
        int id = rs.getInt(1); // Auto-generated ID
        System.out.println("Generated ID: " + id);
    }
}
```

7. Transactions

7.1 What is a Transaction?

- A group of SQL statements that execute as a single unit.
- **ACID:** Atomicity, Consistency, Isolation, Durability.
- If any statement fails, the entire transaction is rolled back.

7.2 Managing Transactions in JDBC

Method	Description
<code>setAutoCommit(false)</code>	Disables auto-commit; starts transaction.
<code>commit()</code>	Commits all changes since last commit/rollback.
<code>rollback()</code>	Rolls back all changes since last commit.
<code>setSavepoint(String)</code>	Creates a savepoint (partial rollback).
<code>rollback(Savepoint)</code>	Rolls back to savepoint.

7.3 Transaction Example

```

try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    conn.setAutoCommit(false); // Start transaction

    // Debit account A
    String debit = "UPDATE accounts SET balance = balance - ? WHERE id = ?";
    PreparedStatement pstmt1 = conn.prepareStatement(debit);
    pstmt1.setDouble(1, 500);
    pstmt1.setInt(2, 1);
    pstmt1.executeUpdate();

    // Credit account B
    String credit = "UPDATE accounts SET balance = balance + ? WHERE id = ?";
    PreparedStatement pstmt2 = conn.prepareStatement(credit);
    pstmt2.setDouble(1, 500);
    pstmt2.setInt(2, 2);
    pstmt2.executeUpdate();

    conn.commit(); // Commit transaction
    System.out.println("Transfer successful");
} catch (SQLException e) {
    // Any exception will rollback automatically if we call rollback in catch
    try { conn.rollback(); } catch (SQLException ex) { ex.printStackTrace(); }
    System.err.println("Transfer failed: " + e.getMessage());
}

```

7.4 Savepoints

```

Savepoint sp = conn.setSavepoint();
// ... some operations
if (condition) {
    conn.rollback(sp); // Rollback only to savepoint
    conn.commit();
}

```

7.5 Isolation Levels

- `Connection.TRANSACTION_READ_UNCOMMITTED`
- `Connection.TRANSACTION_READ_COMMITTED`
- `Connection.TRANSACTION_REPEATABLE_READ`
- `Connection.TRANSACTION_SERIALIZABLE`
- Set via: `conn.setTransactionIsolation(int level);`

8. LeetCode Problems – Approach Discussion

8.1 Rotate Array

- **Problem:** Rotate array to the right by k steps.
- **Approach 1 (Extra array):** Create new array, copy rotated version. $O(n)$ time, $O(n)$ space.
- **Approach 2 (Reverse):**
 - i. Reverse entire array.
 - ii. Reverse first k elements.
 - iii. Reverse remaining $n-k$ elements.
- **Approach 3 (Cyclic replacements):** $O(n)$ time, $O(1)$ space.
- **Key takeaway:** In-place reversal is efficient and avoids extra space.

8.2 Move Zeroes

- **Problem:** Move all zeroes to the end while maintaining relative order of non-zero elements.
 - **Approach (Two-pointer):**
 - i. Maintain `lastNonZeroIndex` = 0.
 - ii. Iterate array; if current $\neq 0$, swap with `arr[lastNonZeroIndex]` and increment.
 - **Time complexity:** $O(n)$, space $O(1)$.
 - **Key takeaway:** This problem tests in-place array manipulation.
-

9. HackerRank: Java JDBC Concepts

- **Task:** Typically conceptual questions about JDBC (not coding).
 - **Topics covered:**
 - Which interface is used to execute SQL queries? (Statement)
 - Which method executes SELECT queries? (executeQuery)
 - How to prevent SQL injection? (PreparedStatement)
 - How to commit a transaction? (Connection.commit())
 - **Skills practiced:** Theory of JDBC components, best practices.
-

10. Corporate Use Case: Enterprise Employee Database Management System

10.1 Problem Context

A large enterprise needs to manage employee records in a relational database.
Requirements:

- Add new employees with auto-generated IDs.

- Update employee details (salary, department).
- Retrieve employees by ID, department, or salary range.
- Delete employees who have left the company.
- Perform batch updates (e.g., annual salary increment).
- Ensure data integrity during salary transfers between departments (transaction).
- Generate reports (e.g., total salary per department).
- Handle database connection issues gracefully.

10.2 Application of Day 10 Topics

Requirement	JDBC Feature Used
Add new employee	<code>PreparedStatement</code> with <code>RETURN_GENERATED_KEYS</code>
Update employee salary	<code>PreparedStatement</code> with parameters
Retrieve employee by ID	<code>PreparedStatement</code> with <code>WHERE id = ?</code>
Retrieve employees by department	<code>PreparedStatement</code> with <code>WHERE department = ?</code>
Get salary range	<code>PreparedStatement</code> with <code>BETWEEN ? AND ?</code>
Delete employee	<code>PreparedStatement</code> <code>DELETE</code>
Batch annual increment	<code>PreparedStatement.addBatch()</code> + <code>executeBatch()</code>
Transfer employee between departments (update dept and salary)	Transaction with <code>setAutoCommit(false)</code> + <code>commit()</code> / <code>rollback()</code>
Total salary per department	<code>PreparedStatement</code> with <code>GROUP BY</code>
Handle connection failures	Try-catch with <code>SQLException</code> , retry logic (optional)
Auto-generate employee ID	<code>RETURN_GENERATED_KEYS</code>
Prevent SQL injection	Always use <code>PreparedStatement</code>
Report generation	<code>ResultSet</code> iteration + aggregation

10.3 Database Schema (Conceptual)

```
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2) NOT NULL,
    department VARCHAR(50) NOT NULL,
    joining_date DATE,
    is_active BOOLEAN DEFAULT TRUE
);
```

```
        active BOOLEAN DEFAULT TRUE
    );
```

10.4 Example Operations (Conceptual Code Snippets)

Add Employee with Generated ID

```
String sql = "INSERT INTO employees (name, salary, department) VALUES (?, ?, ?)";
try (PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
    pstmt.setString(1, "Alice");
    pstmt.setDouble(2, 60000);
    pstmt.setString(3, "Engineering");
    pstmt.executeUpdate();
    ResultSet rs = pstmt.getGeneratedKeys();
    if (rs.next()) {
        int id = rs.getInt(1);
        System.out.println("New employee ID: " + id);
    }
}
```

Transfer Employee (Transaction)

```
conn.setAutoCommit(false);
try {
    // Update department
    String updateDept = "UPDATE employees SET department = ? WHERE id = ?";
    PreparedStatement pstmt1 = conn.prepareStatement(updateDept);
    pstmt1.setString(1, "HR");
    pstmt1.setInt(2, 101);
    pstmt1.executeUpdate();

    // Update salary (adjust for new department)
    String updateSalary = "UPDATE employees SET salary = salary * 1.1 WHERE id = ?";
    PreparedStatement pstmt2 = conn.prepareStatement(updateSalary);
    pstmt2.setInt(1, 101);
    pstmt2.executeUpdate();

    conn.commit();
} catch (SQLException e) {
    conn.rollback();
    System.err.println("Transfer failed: " + e.getMessage());
}
```

Generate Department Salary Report

```
String sql = "SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department";
try (Statement stmt = conn.createStatement()) {
    // Execute the query and process results
}
```

```
ResultSet rs = stmt.executeQuery(sql)) {
    while (rs.next()) {
        System.out.println(rs.getString("department") + ": $" + rs.getDouble("total_salary")
    }
}
```

10.5 Discussion Questions

- Why use `PreparedStatement` instead of `Statement` ? (Security and performance)
- How would you implement pagination for large result sets? (`LIMIT ? OFFSET ?` in SQL)
- What if the database connection fails mid-transaction? (Rollback in catch block)
- How to handle connection pooling in production? (Use HikariCP or Tomcat JDBC pool – not covered today)

11. Hands-on Exercises (Without Code Lines)

11.1 JDBC Connection Flow

Describe the steps to:

- Load the MySQL driver.
- Establish a connection to a local database.
- Execute a SELECT query and process the ResultSet.
- Close all resources.

11.2 PreparedStatement vs Statement

Explain why `PreparedStatement` is preferred for dynamic queries. Give an example of SQL injection attack and how `PreparedStatement` prevents it.

11.3 Transaction Scenarios

For each, describe if a transaction is needed and why:

- Updating a user's email address.
- Transferring money from one account to another.
- Inserting an order and its line items.

11.4 ResultSet Navigation

Describe how to:

- Move to the last row.

- Get the number of rows (not directly – need to iterate or use `COUNT`).
- Retrieve a column by name.

11.5 Error Handling

What exceptions might occur during JDBC operations and how would you handle them?

12. Quiz & Quick Check (Conceptual)

1. Which JDBC component is used to send SQL queries to the database?

- (a) `DriverManager`
- (b) `Connection`
- (c) `Statement`
- (d) `ResultSet`

2. Which method prevents SQL injection?

- (a) `Statement.executeQuery()`
- (b) `PreparedStatement.executeQuery()`
- (c) `ResultSet.next()`
- (d) `Connection.createStatement()`

3. What is the default auto-commit behavior in JDBC?

- (a) `true` (each statement is committed automatically)
- (b) `false` (manual commit required)
- (c) Depends on the database
- (d) Always `false` for SELECT

4. Which method is used to retrieve auto-generated keys after an INSERT?

- (a) `ResultSet.getGeneratedKeys()`
- (b) `Statement.getGeneratedKeys()`
- (c) `PreparedStatement.getGeneratedKeys()`
- (d) `Connection.getGeneratedKeys()`

5. Which isolation level allows dirty reads?

- (a) `TRANSACTION_READ_COMMITTED`
- (b) `TRANSACTION_REPEATABLE_READ`
- (c) `TRANSACTION_READ_UNCOMMITTED`
- (d) `TRANSACTION_SERIALIZABLE`

Answers: 1-c, 2-b, 3-a, 4-c, 5-c

13. Additional Resources & Cheat Sheets

- JDBC URL formats for different databases.
- **PreparedStatement** parameter index – starts at 1, not 0.
- **ResultSet** methods: `next()`, `getXxx(int/string)`, `wasNull()` for null checking.
- **Transaction checklist:** Disable auto-commit, execute operations, commit on success, rollback on failure.

14. Homework & Preparation for Day 11

- **Coding practice (HackerRank):**
 - Review *Java JDBC Concepts* (conceptual).
- **LeetCode:**
 - *Rotate Array* – implement all three approaches.
 - *Move Zeroes* – implement two-pointer solution.
- **Practical:** Set up MySQL locally and connect from Java.
 - Create a sample table.
 - Insert, update, delete, and select using JDBC.
- **Theoretical:** Write a short note on SQL injection and how PreparedStatement prevents it.
- **Reading for Day 11:** Advanced Java Concepts – Generics, Wrapper Classes, Immutable Class, Singleton Design Pattern, SOLID Principles Basics.
- **Self-assessment:**
 - Write a JDBC program to insert a batch of employees (using `addBatch()`).
 - Write a method to get employee by ID using `PreparedStatement`.
 - Explain the difference between `executeQuery()`, `executeUpdate()`, and `execute()`.

15. Closing Summary

Day 10 introduced JDBC, the standard API for database connectivity in Java. Learners understood the architecture, connected to MySQL, and performed CRUD operations using `Statement` and `PreparedStatement`. The importance of `PreparedStatement` for security and performance was emphasized. Transaction management with commit and rollback ensured data integrity for multi-step operations. The Enterprise Employee Database Management System case study demonstrated how JDBC powers real-world applications: adding

employees with auto-generated IDs, updating records, transferring departments with transaction safety, and generating reports. LeetCode problems (Rotate Array, Move Zeroes) provided a break from database concepts and reinforced algorithmic thinking.

16. Key Takeaways:

- JDBC provides a standard interface for database access in Java.
- `PreparedStatement` prevents SQL injection and improves performance.
- Transactions ensure ACID properties; use `commit()` and `rollback()`.
- Always close resources (Connection, Statement, ResultSet) – use try-with-resources.
- CRUD operations are the foundation of database-driven applications.

End of Day 10